

LECTURE 2 • THURSDAY 10 SEPTEMBER 2026

Backpropagation and Loss Functions

computing the gradient, and choosing what to minimise

Carolina Cuesta-Lazaro • PHYS-GA 2043

Transcribed and typeset by Claude from the lecture board notes

1. This week in AI4Science

Links shown in class

- T. Tao, *Mathematics in the age of AI*, public lecture, ICM 2026 (slides) — <https://teorth.github.io/tao-web/slides/age-of-ai-icm-2026.pdf>
- Written up as an essay — <https://arxiv.org/abs/2608.16753>

Tao does not argue about whether AI can do mathematics. He assumes it can, as a working hypothesis, and then asks the question he says the community has been avoiding: what are the goals and values of mathematical research, including the implicit ones?

There are many reasons to do mathematics. To solve open problems, to build theories, to understand the world, to build a community, to train the next generation, to make work of lasting aesthetic value. Tao's central observation is that in the past these goals were positively correlated. Progress on one usually meant progress on the others, so one could be used as a proxy for the rest and most could be left unstated. His worry is that heavy optimisation of a single goal, in particular by AI systems, can pull them apart. Solving many problems very fast no longer implies that anyone understands the solutions, that they have been checked, or that they have been absorbed into the field. He decomposes problem solving into generation, verification, exposition, publication, digestion and canonicalisation, and notes that AI accelerates the first stages while the later ones stay slow and human. The result is a move from proof scarcity to proof abundance, with the bottleneck shifting downstream.

The mechanism he appeals to is Goodhart's law: when a measure becomes a target, it stops being a good measure. A proxy works only while it stays correlated with the thing you actually care about, and optimising hard against the proxy is exactly what breaks the correlation. The same question is worth asking about our own field, since the measures we use in physics are proxies too.

2. How do you compute a gradient?

To run $\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}$ we need the gradient. The question put to the class was how many ways of computing it we knew, and what each would cost. Throughout, write C for the cost of one forward pass $y = f_{\theta}(x)$ and N for the number of parameters. For a modern network N is of order 10^9 .

1. Numerical differentiation

Perturb one parameter at a time and take a finite difference:

$$\frac{\partial \mathcal{L}}{\partial \theta_i} \approx \frac{\mathcal{L}(\theta_i + \varepsilon) - \mathcal{L}(\theta_i)}{\varepsilon}.$$

Each parameter needs its own forward pass, so the whole gradient costs

$$\text{cost} \sim N \cdot C \sim 10^9 C.$$

The accuracy is also poor. The truncation error is of order ε , which argues for small ε , but the numerator is a difference of two nearly equal numbers, so rounding error grows like $\epsilon_{\text{mach}}/\varepsilon$, which argues for large ε . The best you can do is around $\varepsilon \sim \sqrt{\epsilon_{\text{mach}}}$, which leaves roughly half your significant digits. This method is useful for checking a gradient on a small model and for nothing else.

2. Symbolic differentiation

Write the loss as one closed-form expression in θ and differentiate it with the chain rule, as you would by hand:

$$\frac{\partial \mathcal{L}}{\partial \theta_L}(\theta_L), \quad \frac{\partial x_L}{\partial \theta_{L-1}}(\theta_{L-1}), \quad \dots$$

This is exact, but the expression for the derivative is much larger than the expression for the function, and it grows with depth. The same subexpressions appear many times and get recomputed. For a network of any realistic size the formula cannot be written down, let alone evaluated efficiently.

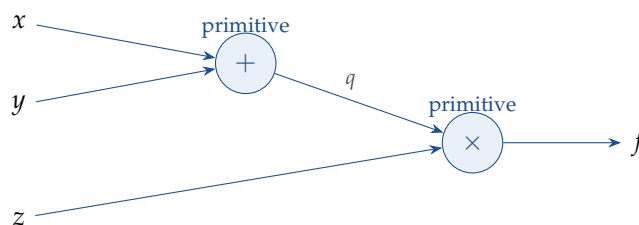
3. Automatic differentiation

Automatic differentiation computes the full gradient, with respect to all N parameters, at a cost of about 2–3 C . The cost does not depend on N .

This is the result that makes deep learning possible. Training a network with 10^9 parameters costs roughly three forward passes per step, not 10^9 of them.

3. Autodiff and the computational graph

Automatic differentiation is neither numerical nor symbolic. The idea is to stop treating the function as one large formula and instead treat it as a graph of small operations, called *primitives*, each of which we know how to differentiate.



The graph above computes

$$q = x + y, \quad f = q \cdot z.$$

Each primitive has a local derivative we know in closed form: $\partial q / \partial x = \partial q / \partial y = 1$, $\partial f / \partial q = z$ and $\partial f / \partial z = q$. The derivative of the whole computation with respect to any input is then a product of local derivatives along the path connecting them:

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial y} = z.$$

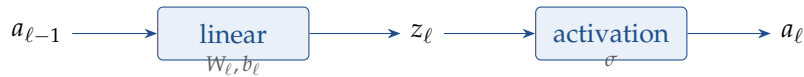
Backpropagation applies this rule once, working backwards from the loss through the graph. Each intermediate derivative is computed once and reused by everything downstream of it, rather than being recomputed for every parameter. That reuse is why the whole gradient costs a small multiple of a single forward pass.

4. Backpropagation in a deep network

Write a network of L layers as

$$z_\ell = W_\ell a_{\ell-1} + b_\ell, \quad a_\ell = \sigma(z_\ell), \quad \ell = 1, \dots, L,$$

with $a_0 = x$. Each layer does a linear map followed by an elementwise activation.



Let $\delta_\ell = \partial \mathcal{L} / \partial z_\ell$ and let $D_\ell = \text{diag}(\sigma'(z_\ell))$, the derivative of the activation at layer ℓ . Applying the chain rule layer by layer gives the backward recursion

$$\delta_L = D_L \frac{\partial \mathcal{L}}{\partial a_L}, \quad \delta_\ell = D_\ell W_{\ell+1}^\top \delta_{\ell+1}, \quad \frac{\partial \mathcal{L}}{\partial W_\ell} = \delta_\ell a_{\ell-1}^\top.$$

Unrolling the recursion shows what the gradient at layer ℓ actually is:

$$\delta_\ell = D_\ell W_{\ell+1}^\top D_{\ell+1} W_{\ell+2}^\top \cdots W_L^\top D_L \frac{\partial \mathcal{L}}{\partial a_L}.$$

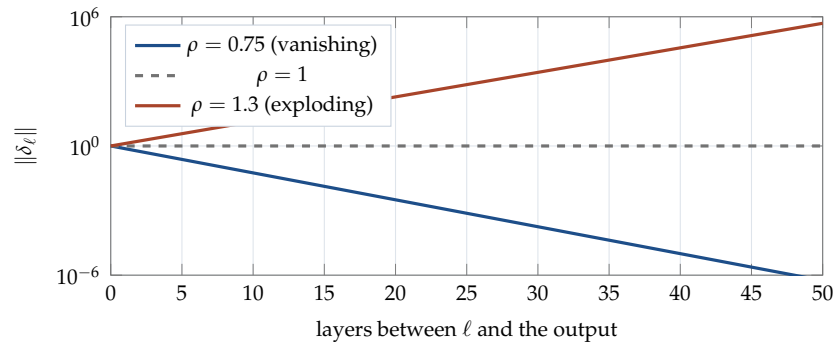
It is a product of $L - \ell$ matrix factors, one pair per layer between ℓ and the output. That product is where the next problem comes from.

5. Vanishing and exploding gradients

Take the typical size of one factor $D_k W_k^\top$ to be ρ . Then the size of the gradient reaching layer ℓ behaves like

$$\|\delta_\ell\| \sim \rho^{L-\ell}.$$

An exponential in depth has only two behaviours, and neither is what we want.



If $\rho < 1$ the gradient vanishes. The early layers receive essentially nothing and stop learning, so a deep network trains only in its last few layers. If $\rho > 1$ the gradient explodes, the updates are enormous and training diverges. Only $\rho \approx 1$ works, and the deeper the network the more precisely that has to hold.

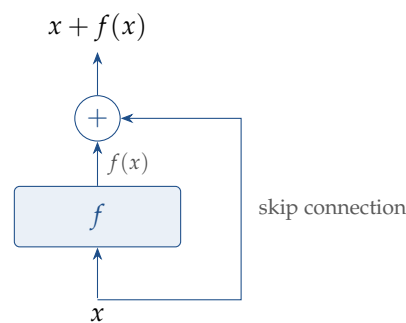
The sigmoid makes this worse. We saw in Lecture 1 that $\sigma' \leq \frac{1}{4}$, so every D_k contributes a factor of at most a quarter and vanishing is the default outcome. This is one of the reasons ReLU, whose derivative is 0 or 1, replaced it.

Careful initialisation, normalisation layers and gradient clipping all help, but they are adjustments to the size of the factors. The structural fix is to change what is being multiplied.

6. Residual connections

Add the input of a block to its output:

$$a_\ell = a_{\ell-1} + f_\ell(a_{\ell-1}), \quad \text{with, for example, } f_\ell(a) = A(Wa + b).$$



The Jacobian of one block is now

$$\frac{\partial a_\ell}{\partial a_{\ell-1}} = \mathbb{I} + J_{f_\ell},$$

and the Jacobian of the whole network is the product of these. Expanding that product is what matters:

$$\prod_\ell (\mathbb{I} + J_{f_\ell}) = \mathbb{I} + \sum_\ell J_{f_\ell} + \sum_{\ell < \ell'} J_{f_{\ell'}} J_{f_\ell} + \dots$$

Instead of a single product of L factors, which is exponentially small or exponentially large, the gradient is now a sum over all subsets of layers. The first term is the identity: a path from the loss back to any layer that passes through no Jacobian at all. The gradient can no longer vanish simply because the network is deep.

There is a second reading of the same formula. At initialisation f_ℓ is small, so each block is close to the identity and a deep residual network starts out behaving like a much shallower one, becoming effectively deeper as training proceeds. This is why residual networks train at hundreds of layers, and why essentially every modern architecture, transformers included, is built from residual blocks.

Do they help with exploding gradients too?

Only indirectly, and it is worth being clear about why. Taking norms in the product above,

$$\left\| \prod_{\ell} (\mathbb{I} + J_{f_\ell}) \right\| \leq \prod_{\ell} (1 + \|J_{f_\ell}\|) = (1 + j)^L \quad \text{if } \|J_{f_\ell}\| \sim j.$$

This is still exponential in depth, and the base is now at least 1. Residual connections put a floor under the gradient but no ceiling on it: they remove the vanishing failure mode and leave the exploding one. The same is true on the forward pass, where $a_\ell = a_{\ell-1} + f_\ell(a_{\ell-1})$ makes the activations accumulate and grow with depth.

What they do change is where the safe setting lies. In a plain network the signal has to pass through every W_ℓ , so the weights must be tuned so that $\rho \approx 1$: too small and the signal dies, too large and it blows up, and the target gets narrower with depth. In a residual network the identity already carries the signal, so the safe configuration is simply J_{f_ℓ} small, with each block doing very little at first. Making a residual branch small is easy, for instance by initialising its last layer near zero or scaling it by $1/\sqrt{L}$. Making a product of matrices have singular values close to 1 is not.

Exploding gradients are then dealt with separately: normalisation layers inside the block to hold the activations at a fixed scale, and gradient clipping, which rescales the update whenever its norm exceeds a threshold.

7. What loss should we use?

So far $\mathcal{L}$ has been an unspecified scalar. We now ask what to put there.

Mean squared error

The obvious choice is to penalise being wrong, and to penalise being more wrong more:

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2, \quad \hat{y}_i = f_\theta(x_i).$$

This is the mean squared error, also called the L2 loss. It is smooth, differentiable everywhere, always non-negative, and zero only when the prediction is exactly right. For a linear model it is convex with a closed-form minimum. It is the default for regression and it is what most people reach for without thinking about it.

L1, and what squaring does to outliers

An equally reasonable choice is to penalise the error linearly:

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N |\hat{y}_i - y_i|.$$

The two behave very differently in the presence of bad data. Under L2 an error of 10 counts a hundred times an error of 1, and its gradient contribution grows with the error, so a single mislabelled point can dominate the total gradient and pull the fit towards itself. Under L1 the derivative of $|\cdot|$ is ± 1 however large the error is, so one bad point contributes no more than any other. L1 is robust to outliers; L2 is not.

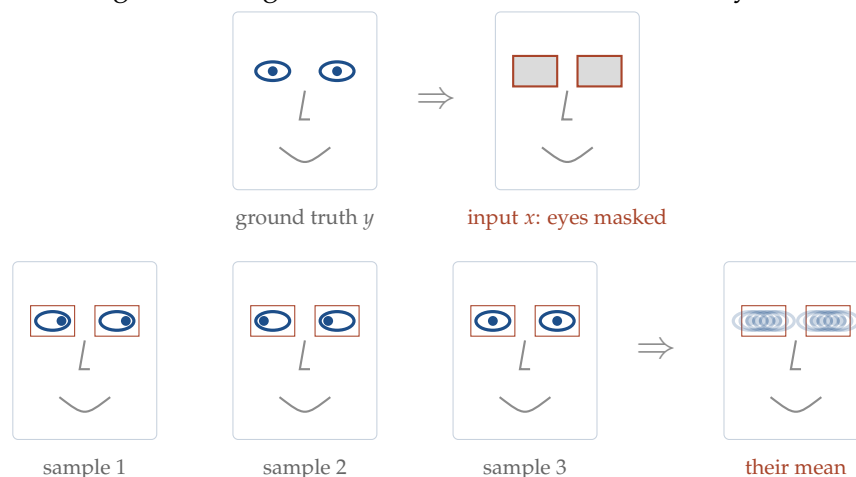
The price of L1 is that it is not differentiable at zero, and that its gradient does not shrink as you approach the minimum, so the step size has to be decayed by hand.

So we have two defensible losses that give different answers on the same data. Choosing between them by trial and error would be unsatisfying, and it turns out to be unnecessary: these are not arbitrary ways of measuring distance, and each of them is answering a specific question.

A problem where the answer is not unique

Start from a real photograph of a face. Mask out the two boxes containing the eyes and hand what is left to a network as its input x . The eyes are now gone, and many different eyes are consistent with the rest of the face: looking left, looking right, open, half closed. Nothing in the visible pixels picks one out.

If we could draw samples from the set of consistent completions, we would get a sequence of sharp faces whose eyes move around inside those boxes. Train the same network with mean squared error and we get one image, with a blurred smear where the eyes should be.



This is a documented effect and not an artefact of a weak network. In the super-resolution literature the same observation was made explicitly: minimising the mean squared error drives the model towards a pixel-wise average of the plausible solutions, which is smooth and perceptually poor.¹

¹C. Ledig et al., *Photo-realistic single image super-resolution using a generative adversarial network*, arXiv:1609.04802; their Figure 3 makes exactly this point.

The lesson is that y is not a function of x here. What the data define is a conditional distribution

$$p(y | x),$$

the distribution over labels given the input.

Prior and posterior. $p(y)$ is the *prior*: what images look like before we observe anything, which the network learns implicitly from the training set. It is the prior that knows faces have two roughly symmetric eyes. Observing x , the unmasked pixels, updates it to the *posterior* $p(y | x)$: the completions consistent with what we can see, weighted by how plausible they were to begin with. Without a prior the problem is hopeless, since any pixels at all are consistent with a masked region.

If the likelihood were Gaussian

Suppose we model that distribution as a Gaussian centred on the network output,

$$p(y | x) = \mathcal{N}(y; f_\theta(x), \sigma^2),$$

which says the label is the prediction plus symmetric noise of the same width everywhere. Its negative log-likelihood is

$$-\log p(y | x) = \frac{(y - f_\theta(x))^2}{2\sigma^2} + \log(\sigma\sqrt{2\pi}).$$

With σ fixed, the second term is a constant and the first is the squared error up to a factor. Setting $\sigma = 1$,

$$-\log p(y | x) = \frac{1}{2}(y - \hat{y})^2 + \text{const},$$

so minimising mean squared error is maximum likelihood under a Gaussian of unit width. The same calculation with a Laplace distribution in place of a Gaussian gives the L1 loss. The two losses from the start of this section are the negative log-likelihoods of two different assumptions about the noise, which is the reason they disagree.

The result does not need that assumption

It would be reasonable to conclude that the blurry face is the fault of the Gaussian assumption, and that the fix is to assume something better. It is not. The derivation below makes no assumption at all about $p(y | x)$ beyond its having a mean and a finite variance.

Train with

$$\mathcal{L}(\theta) = \mathbb{E}_x \mathbb{E}_{y \sim p(y|x)} [(\hat{y} - y)^2], \quad \hat{y} = f_\theta(x).$$

Fix one value of x and look at the inner expectation. With x fixed, $\hat{y}$ is a number, and the only random quantity is y . Define the mean of the posterior

$$\mu(x) = \mathbb{E}_{y \sim p(y|x)}[y] = \int y p(y | x) dy.$$

Now add and subtract μ inside the square and expand:

$$\begin{aligned}\mathbb{E}_y[(\hat{y} - y)^2] &= \mathbb{E}_y[(\hat{y} - \mu) + (\mu - y)]^2 \\ &= \underbrace{\mathbb{E}_y[(\hat{y} - \mu)^2]}_{(i)} + 2 \underbrace{\mathbb{E}_y[(\hat{y} - \mu)(\mu - y)]}_{(ii)} + \underbrace{\mathbb{E}_y[(\mu - y)^2]}_{(iii)}.\end{aligned}$$

Take the three terms in turn.

- (i) Neither $\hat{y}$ nor μ depends on y , so the expectation does nothing and this term is just $(\hat{y} - \mu)^2$.
- (ii) Pull the constant factor $2(\hat{y} - \mu)$ out of the expectation, leaving $\mathbb{E}_y[\mu - y] = \mu - \mathbb{E}_y[y] = \mu - \mu = 0$. The cross term vanishes, and it vanishes precisely because μ was chosen to be the mean.
- (iii) This is $\mathbb{E}_y[(y - \mu)^2] = \text{Var}[y | x]$, the spread of the posterior. It does not contain θ at all.

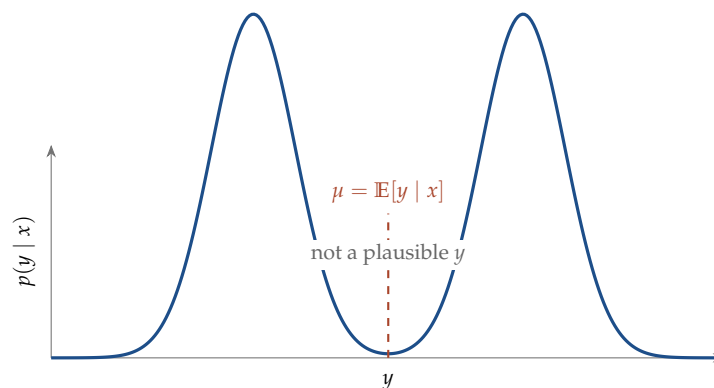
$$\mathbb{E}_{y \sim p(y|x)}[(f_\theta(x) - y)^2] = \underbrace{(f_\theta(x) - \mu(x))^2}_{\text{we control this}} + \underbrace{\text{Var}[y | x]}_{\text{we do not}}$$

The second term is a floor that no model can go below. The first is minimised, and equals zero, when

$$f_\theta(x) = \mu(x) = \mathbb{E}[y | x].$$

Nothing in that calculation assumed the posterior was Gaussian. Whatever $p(y | x)$ actually is, bimodal or skewed or heavy-tailed, a network trained with mean squared error converges to its mean. It is not learning to produce a plausible y ; it is learning to produce the average of all plausible y . For the masked face, the average of many sharp faces whose eyes sit in different places is a blur. The blur is not a failure of the network. It is the correct answer to the question we asked.

It is worth being explicit that the mean of a distribution need not be a plausible value drawn from it. If the posterior has two modes, the mean sits between them, at a value the posterior itself considers unlikely:



The same derivation with an absolute value in place of a square returns the *median* of the posterior instead. That is the same robustness we saw earlier, seen from the other side: the median ignores how far away the outliers are, and the mean does not. But the median is still a single point estimate, so L1 does not solve the underlying problem either.

Other losses

Most of the losses we use are negative log-likelihoods of some assumed $p(y | x)$, and each returns a different summary of the posterior.

Loss	Assumed $p(y x)$	What the optimum returns
$(\hat{y} - y)^2$ (L2, MSE)	Gaussian, fixed width	mean of the posterior
$ \hat{y} - y $ (L1)	Laplace	median of the posterior
Cross entropy	categorical	the full posterior over classes
KL divergence, Wasserstein	—	compare distributions, not points
Denoising loss	—	the basis of diffusion and flow matching

Cross entropy is the interesting exception. For a discrete label the network can output a probability for every class, so it represents the whole posterior rather than one number summarising it. That is only possible because the label takes finitely many values.

Where this leaves us

We now have the gradient, at a cost that does not grow with the number of parameters, and residual connections to keep it from vanishing as the network gets deeper. The remaining lesson is about what we ask the network to do. If the answer is genuinely uncertain, a single output trained with mean squared error gives one summary statistic of the posterior and has no way to express its spread. To obtain sharp, plausible answers we have to model $p(y | x)$ itself and sample from it, which is what generative models do and where the course goes later in the term.

Next lecture we turn to inductive biases and architectures: the MLP, convolutional networks, graph networks and transformers.

Due Thursday 17 September. Literature review: read between five and ten papers in an area you are interested in, and for each write one sentence on the main idea, one on something you would like to understand better, one on an experiment you could run to build on it, and one on where you think there is a gap. Form your team.